Who Will Be Late? Prioritising Delivery-Risk Review at Order Approval

A Leakage-Safe PostgreSQL Pipeline, a Frozen Temporal Evaluation and a Review-Capacity Simulation on 99,441 Olist Marketplace Orders

Code: GitHub Repository | App: local Dash application (Section 10)

Yassine Erraji
October 2026

Abstract This report documents an end-to-end analytics system built on the public Olist marketplace export (99,441 orders, 2016–2018) to answer one operational question: when an order is approved, which orders are most likely to arrive after the date promised to the customer, and how should a team with a fixed review capacity prioritise them? Every transformation, from typed staging with quarantine to point-in-time features, is written in PostgreSQL and guarded by 45 assertions, 24 of which block publication. On the real data, a naive join of items, payments and reviews silently inflates item value by 4.0%; the published tables reconcile exactly. Seller histories only use deliveries that had already happened before approval, and an independent Python re-implementation reproduces all 99,991 order–seller rows. Models are compared under a frozen four-block temporal protocol in which an outcome becomes usable only once the parcel is delivered. On a 13-week test block (19,055 delivered orders, 3.8% late), the logistic regression chosen beforehand reaches an average precision of 0.075, against 0.043 for a seller-history rule (paired weekly-bootstrap difference +0.032, 95% interval +0.015 to +0.060) and 0.038 without skill. Reviewing 10% of each day's approvals would have put 8.2% late orders in the queue instead of 3.8%, capturing 22.0% of them. The report is equally explicit about what did not work: calibration learned on a disrupted period over-estimates risk later, model selection saw only 274 of 1,287 late examples because slow parcels arrive after the cut-off, and a LightGBM not selected in advance did better on test. Nothing here shows that reviewing an order prevents a delay; Section 11 describes the randomised pilot that would.

1 Introduction

A marketplace shows a delivery date at checkout and then depends on many independent sellers and carriers to keep it. When the promise is broken, customers notice: in this dataset, orders delivered late receive a mean review score of 2.27 out of 5, against 4.29 for orders delivered on time, and 62% of late orders get a score of 1 or 2 (9% of on-time orders). An operations team can intervene before the delivery, for instance by contacting a seller or adjusting a promise, but only for a small share of the orders it receives every day.

The decision studied here is therefore a *ranking under capacity*: at the moment an order is approved, sort the day's orders by estimated risk of late delivery and look at the top q%. Two properties make this harder than a standard classification exercise. *The data is relational*: an order can contain several items from several sellers, several payments and several reviews, so any careless join multiplies rows. *The data is temporal*: the outcome of an order is only known once it is delivered, and the source is a final export rather than a versioned event log, so it is easy to let information from the future leak into a feature or into model selection.

The contribution is less a model than an *evaluation discipline* applied end to end:

- a PostgreSQL pipeline whose grains, keys and totals are asserted before anything is published (Section 3);
- point-in-time seller histories computed on delivery events, verified by mutation tests and an independent re-implementation (Section 5);
- a frozen temporal protocol with label-availability cut-offs, two baselines and paired block bootstrap intervals (Sections 6–7);
- a daily review-capacity simulation that separates what a reviewer would have seen from what can be evaluated afterwards (Section 8);
- a measured SQL performance study and a read-only analytics application (Sections 9–10).

2 Data and Conventions

The source is the *Brazilian E-Commerce Public Dataset by Olist*, version 2 on Kaggle (CC BY-NC-SA 4.0) [1]: nine CSV files describing orders placed between September 2016 and October 2018. Each file is loaded as text with its batch, file name and physical line number, and a manifest records its size, SHA-256 and record count (Table 1); re-importing an identical file is a no-op.

Conventions. Prediction time is the approval timestamp; an order without approval is not scored, and its purchase time is never used as a substitute. An order is *late* when its delivered *date* is after its promised date. Dates rather than timestamps are compared because promises are stored at midnight: a parcel delivered at 15:00 on the promised day is on time. Timestamps carry no time zone in the source and none is invented. Item value, freight and payments are three distinct measures, none of which is Olist's revenue or margin.

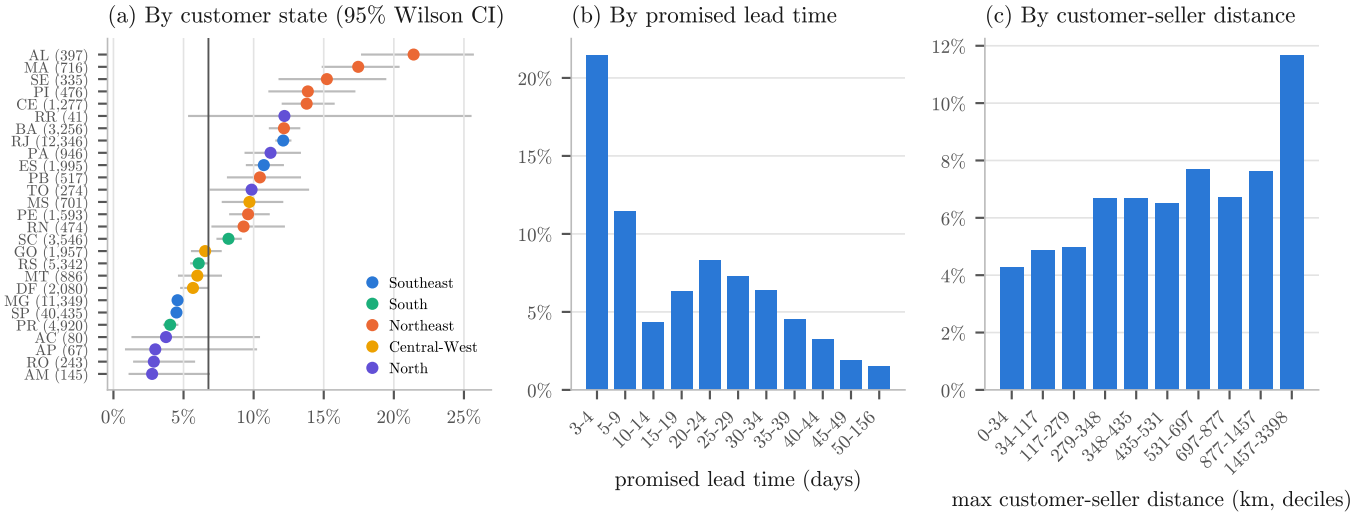


Figure 1: Late rate among eligible delivered orders. (a) Customer states with at least 30 orders, 95% Wilson intervals, coloured by macro-region; the vertical line is the overall rate. (b) Promised lead time (purchase to promised date). (c) Deciles of the largest customer–seller distance between zip-prefix centroids.

Table 1: Source files (current load). Records exclude headers; review messages span several physical lines, which the loader reassembles.

File	Records	MB	SHA-256
`geolocation`	1,000,163	61.3	`b514f6fc`
`order_items`	112,650	15.4	`0bc4d068`
`order_payments`	103,886	5.8	`4f713964`
`customers`	99,441	9.0	`983a4222`
`orders`	99,441	17.7	`8df58ef3`
`order_reviews`	99,224	14.5	`012b61c7`
`products`	32,951	2.4	`3e656962`
`sellers`	3,095	0.2	`1f643d2b`
`category_translation`	71	0.0	`a81f0d1f`

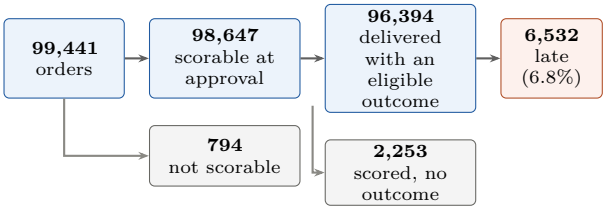


Figure 2: Outcome populations. Not scorable: e.g. 160 orders without approval and orders without items. Scored without outcome: 467 canceled or unavailable, the rest still in transit at export. Features and scores exist for every scorable order; performance can only be measured on the labelled ones.

Populations. Canceled, unavailable and undelivered orders are not "on time"; they form separate populations (Figure 2). An order is *scorable* if the information available at approval is coherent (approval present, promise not before purchase or approval, at least one item). It is *labelled* if, in addition, a delivery was observed after the approval. Every exclusion carries a reason and appears in a coverage table.

3 A Relational Pipeline in PostgreSQL

Figure 3 summarises the layers. *Staging* casts every field with ordered `CASE` checks (a cast only runs after its format check passed, which an `OR` would not guarantee) and `pg_input_is_valid`, so a malformed date is quarantined with a reason instead of aborting the load. Rows sharing a key are split into strict duplicates (kept once and logged) and key conflicts (all versions quarantined, publication blocked). *Core* then adds primary and foreign keys only after these checks. A child row whose parent was quarantined is cascaded with a warning, and one whose parent does not exist at all is a critical orphan.

On the real files no row failed typing, no key conflicted and no reference was orphaned. All 18 critical transform checks passed, and the anomalies that do exist are counted rather than hidden: 775 orders without items, 160 without approval, 1,359 carrier hand-offs recorded before approval, 610 products without category and 303 orders whose payments differ from items plus freight.

The fan-out trap. Items, payments and reviews each hang under the order with their own grain (9,803 orders have several items, 2,961 several payments, 547 several reviews). Joining them directly on `order_id` multiplies rows (Figure 4). On the real data the naive join returns 117,329 rows instead of 112,650 items and reports R$14.14 M of item value instead of R$13.59 M: a 4.0% error, small enough to go unnoticed in a dashboard. The marts aggregate each child to the order grain first and join one-to-one; an assertion compares every total before and after the join.

Multi-seller orders. 1,278 orders involve more than one seller, but the delivery timestamp exists only at order level. A seller's *late-order exposure* therefore counts each

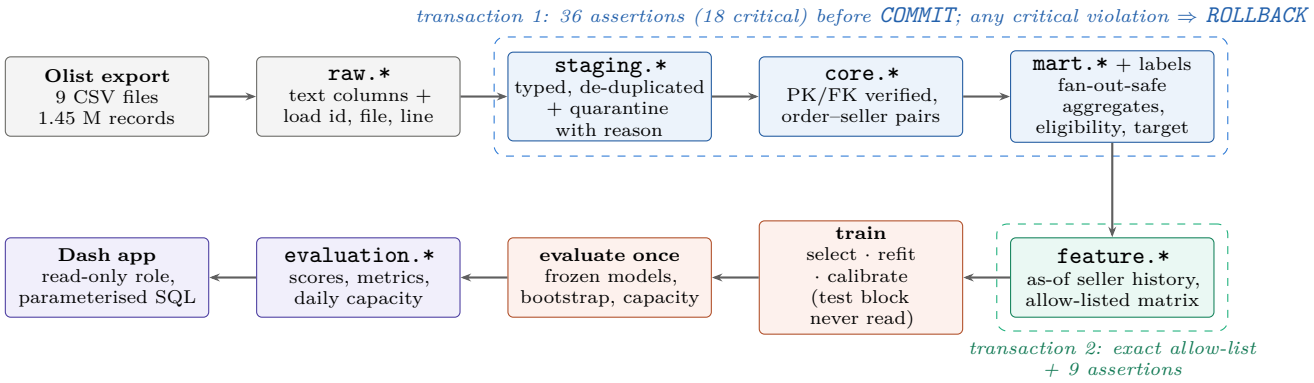


Figure 3: Pipeline. Each layer is rebuilt inside one transaction; because PostgreSQL DDL is transactional, readers keep seeing the previous tables until the assertions pass. The dashboard reads only `mart`, `evaluation`, `feature`, `quality` and `meta` with a read-only role.

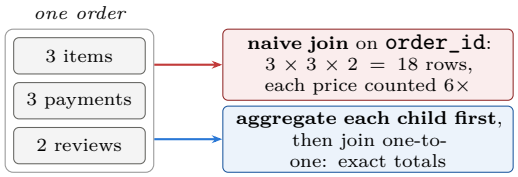


Figure 4: The fan-out trap on one test-fixture order (130.00 of items becomes 780.00). On the real data the same mistake inflates item value by 4.0%.

order once per seller through a distinct order–seller table; it is not a measure of seller-caused delay, and seller counts do not add up to the marketplace total.

4 What the Operational Data Shows

Of the 96,394 delivered orders with an eligible outcome, 6,532 were late (6.8%). Lateness is concentrated in time and space (Figure 1). It peaks with the November 2017 Black Friday volume (12.2% by approval month) and in February–March 2018 (13.6% and 19.2%), then falls to 1.2% in June 2018. Customers in the Northeast are late 12.7% of the time, against 6.1% in the Southeast. The rate also rises with customer–seller distance, from 4.3% in the nearest decile to 11.7% beyond 1,457 km. Promise length works the other way: very short promises (3–4 days, n =331) are broken 21.5% of the time, whereas promises of 50 days or more almost never are (1.5%). These are associations computed on delivered orders. They describe where risk sits; they do not say that distance or promise length *causes* delays.

5 Point-in-Time Features

The matrix contains 27 allow-listed features in four groups (Table 2). Nothing observed after approval enters it, the classic source of leakage [7]: no delivery date, no carrier hand-off, no status, no review, no payment (payments lack a reliable availability log) and no raw identifier. The allow-list is checked three times: when the matrix is built (an exact column comparison rolls the build back), by a SQL assertion on column names, and by unit tests.

Table 2: Feature groups. "Assumed" means plausible at approval but unverifiable in a final snapshot.

Group	#	Examples	At approval
Order	11	promised lead days, item value, freight ratio, approval delay, shipping limit	assumed
Geography	4	customer state, seller state, same-state flag, distance	assumed
Catalog	4	main category, weight, volume, # categories	assumed
Seller history	8	known outcomes, 90-day late rate, recent volume, no-history flag	observed

The as-of rule. For an order approved at t, a past order of the same seller contributes an outcome only if it was *delivered* strictly before t (Figure 5). An order bought before t but still in transit at t contributes nothing, and a delivery at exactly t is excluded. The 90-day window is anchored on delivery events, and approval-volume counters use approvals in $[t - 30\,\mathrm{d}, t)$. A moving average of final labels ordered by purchase date would violate this rule, because it uses outcomes that were not yet known. Per-seller histories are averaged with equal weight over the order's *distinct* sellers, so a seller listed on two items counts once.

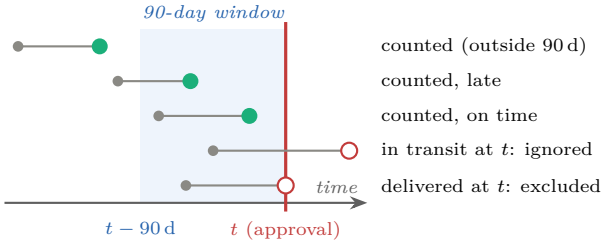


Figure 5: Which past orders of a seller inform the score of an order approved at t. Each line runs from a past order's approval (grey dot) to its delivery: green if strictly before t, red otherwise.

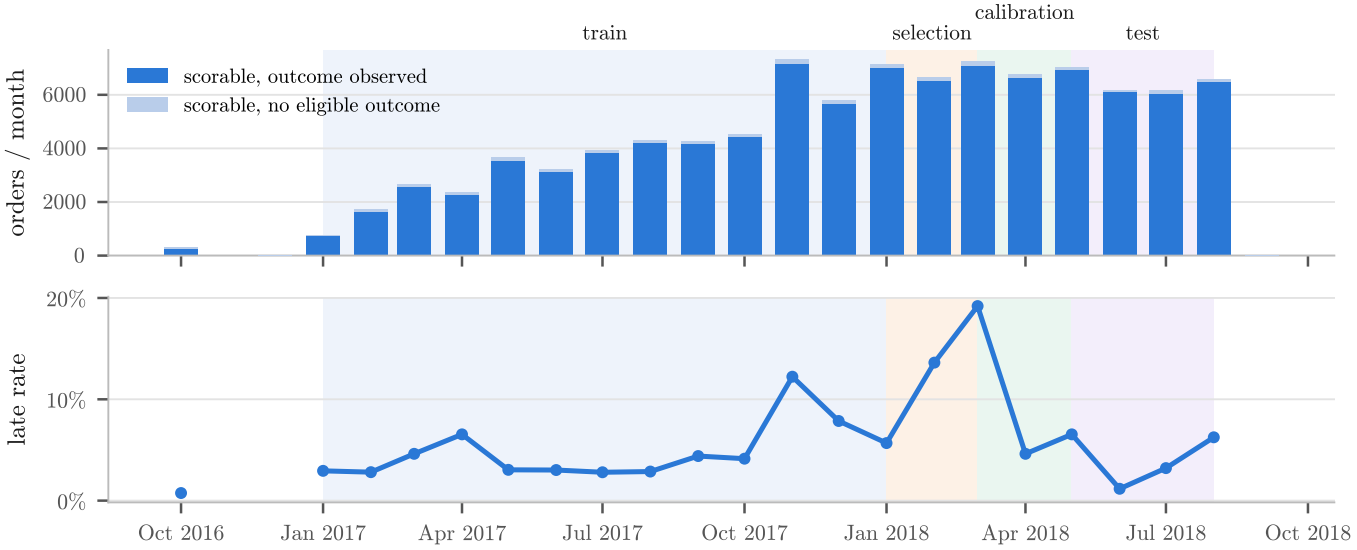


Figure 6: Scorable orders per approval month (light: no eligible outcome) and late rate, with the four temporal blocks shaded. The calibration block falls in a disrupted period (late rate up to 19.2%), whereas the test block is calm (1.2% in June).

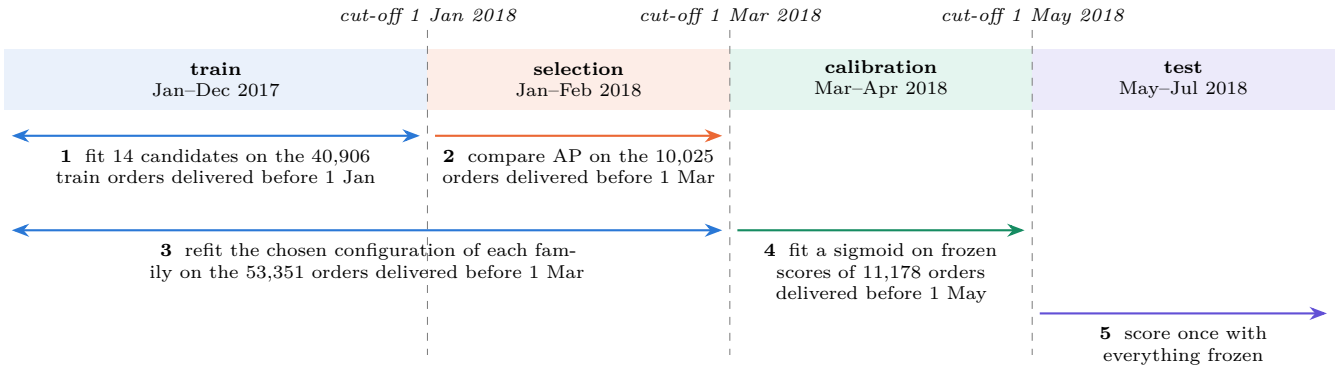


Figure 7: Training protocol (not to scale). At every step only labels whose delivery happened before the next cut-off are used; models, preprocessing and calibrators are frozen before the test block is scored.

Table 3: Temporal blocks on the approval date ([start, end)). The last two columns count the labelled orders, and the late ones among them, that were already delivered at the next cut-off and could therefore be used.

Block	Start	End	Scorable	Labelled	Late	Late rate	Usable at cut-off	Late usable
Train	2017-01-01	2018-01-01	44,507	43,345	2,452	5.7%	40,906	1,762
Selection	2018-01-01	2018-03-01	13,815	13,525	1,287	9.5%	10,025	274
Calibration	2018-03-01	2018-05-01	14,041	13,707	1,666	12.2%	11,178	1,243
Test	2018-05-01	2018-08-01	19,368	19,055	718	3.8%	all observed	

Verification. Hand-written fixtures pin the expected values of each rule. One test moves a delivery from exactly t to one second earlier and checks that it then counts. Mutation tests rewrite future deliveries and reviews, or add future orders, and assert that every feature of earlier orders is byte-identical. Finally, an independent Python implementation was run against the SQL on the full real dataset and agreed on all 99,991 order–seller rows.

A known bias remains: histories describe *delivered* orders, so a seller whose parcels are currently stuck looks better than it is until they arrive.

6 Temporal Evaluation Protocol

Four contiguous calendar blocks are defined on the approval date (Figure 6, Table 3). Their boundaries were proposed from public coverage information and confirmed unchanged by a coverage audit of the downloaded files, *before any model was trained*. The audit reports volumes, label coverage, label delays and late counts per month and per block; label coverage is at least 95.1% in every month of the blocks (lowest in January 2017). The definition is hashed (`64ef33ebf2ae`) and training refuses any block definition that is not frozen.

Table 4: Test block (May 1, 2018 to August 1, 2018, exclusive): 19,055 delivered orders with an eligible outcome, 718 late (prevalence 3.8%). ⋆Selected on the selection block before the test was scored. Brier scores before and after the sigmoid calibrator; ranking metrics are unchanged by calibration.

Model	AP	95% CI	ROC-AUC	Precision@10%	Recall@10%	Lift@10%	Brier raw	Brier calibrated
Constant (prevalence)	0.038	[0.021, 0.058]	0.500	3.0%	7.9%	0.79	0.0364	0.0364
Seller-history rule	0.043	[0.021, 0.066]	0.536	5.1%	13.6%	1.36	0.0482	0.0417
Logistic regression⋆	0.075	[0.047, 0.115]	0.708	8.7%	23.0%	2.30	0.2520	0.0455
LightGBM	0.090	[0.070, 0.115]	0.700	10.4%	27.7%	2.77	0.0361	0.0435

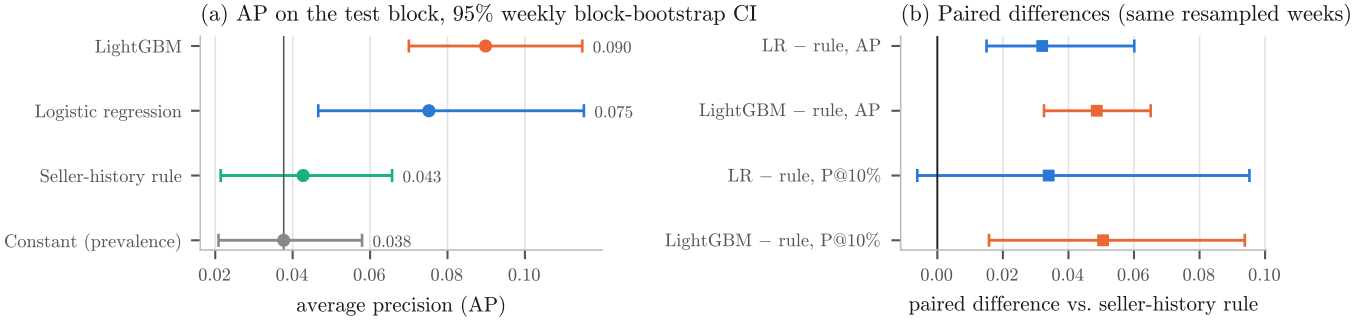


Figure 8: (a) Average precision with 95% weekly block-bootstrap intervals; the vertical line is the no-skill value (prevalence). (b) Paired differences against the seller-history rule, computed on the same resampled weeks.

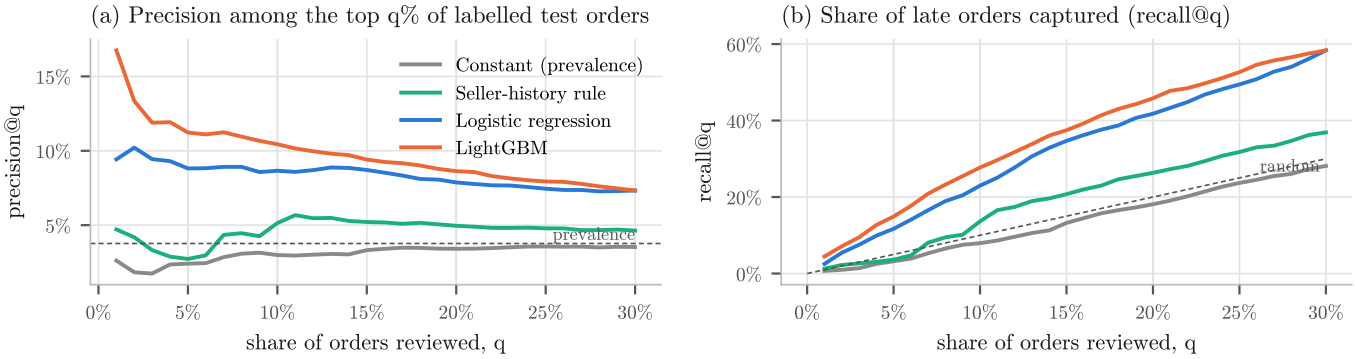


Figure 9: Global top-q curves on the labelled test orders. The dashed lines are the prevalence (a) and random selection (b); vertical guides mark 5, 10 and 20%.

Label availability. An outcome is usable from the actual customer delivery, including for late orders. Figure 7 shows the five steps. Candidates are fitted on train orders delivered before the selection cut-off. A fixed grid (4 logistic regressions, 8 LightGBM [5] configurations, written down before any result) is compared by average precision on selection orders delivered before the calibration cut-off. The chosen configuration of each family is refitted on train plus selection, and a sigmoid calibrator is fitted on the frozen scores of the calibration block. The test block is then scored once. The cut-offs have a cost, visible in Table 3: late parcels arrive after on-time ones, so when the model was chosen only 274 of the 1,287 late selection orders had been delivered. This is what a real deployment would have known at that date, and it makes model choice noisy.

Metrics. The headline metric is average precision (AP, scikit-learn's step-wise definition [8]), which focuses on the top of the ranking and whose no-skill value is the prevalence; with 3.8% positives, ROC-AUC would flatter weak rankings [2]. Precision, recall and lift at q use $k = \lceil qN \rceil$ with ties broken by order id. Uncertainty comes from a paired bootstrap that resamples whole ISO weeks of the test block (500 resamples, 14 weeks) [6], so that every model is scored on the same resampled weeks. Every evaluation is logged; re-running it on the same run is recorded as a verification, and a model trained after a test evaluation would be flagged as exploration.

7 Results

The model beats the rule on AP, modestly. The selected logistic regression ($C = 0.1$, class weight balanced) reaches an AP of 0.075 on the test block, against 0.043 for the seller-history rule and 0.038 for a constant score (Table 4). The paired difference with the rule is +0.032 (95% interval +0.015 to +0.060) and is positive in every resample (Figure 8). At the 10% operating point the picture is less clear-cut: precision rises from 5.1% to

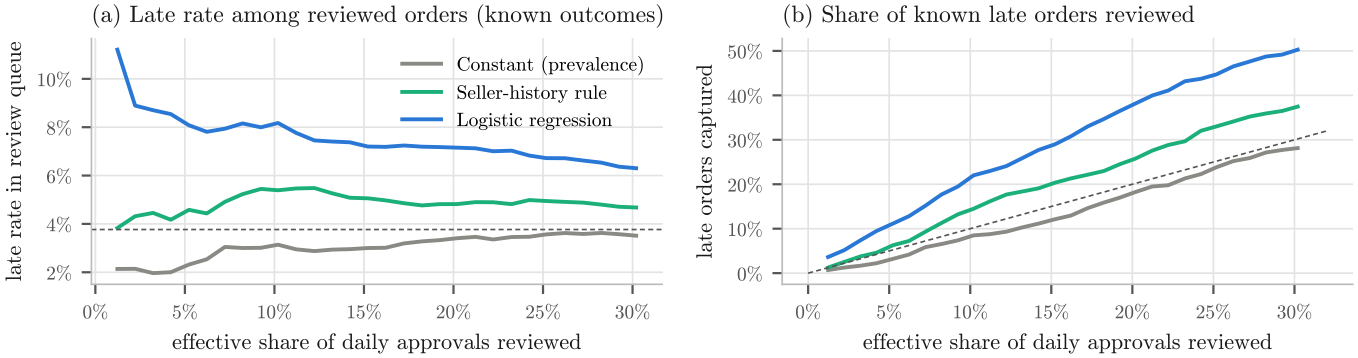


Figure 10: Daily review policy over the 91 test days. (a) Late rate among reviewed orders with a known outcome; the dashed line is the late rate of all known orders. (b) Share of all known late orders that were reviewed; the dashed diagonal is random selection.

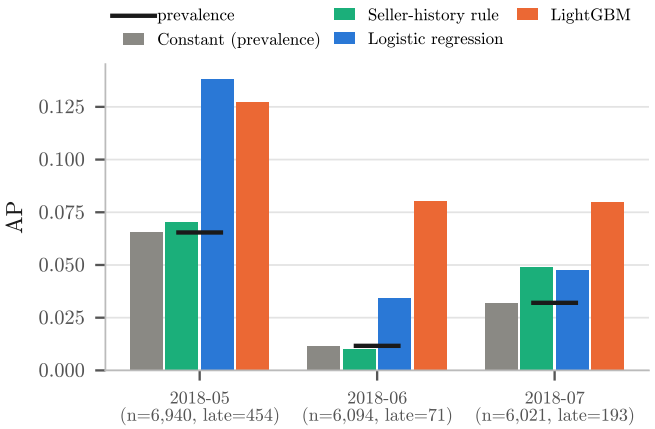


Figure 11: AP by test month; black ticks mark the month's late rate (no-skill AP).

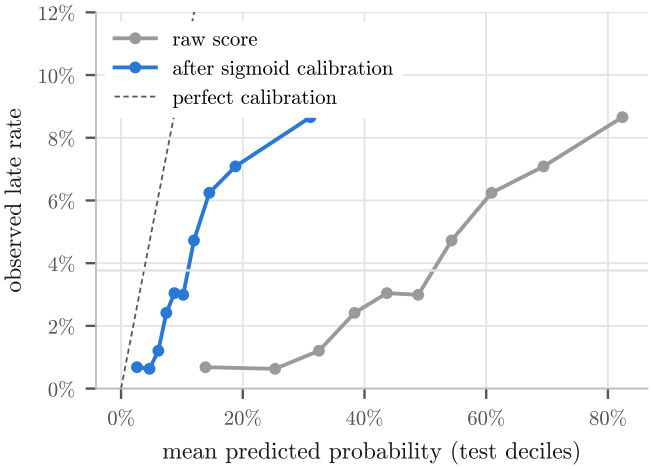


Figure 12: Calibration of the selected model on test deciles. The calibrator learned an 11.1% late rate; the test rate is 3.8% (horizontal line).

8.7%, a lift of 2.30 over the prevalence, but the paired difference (+3.4 points, interval -0.6 to +9.5) includes zero. The absolute level stays modest: more than nine reviewed orders in ten would have arrived on time anyway.

The model selected in advance was not the best one. LightGBM, which lost the selection (selection AP 0.055 vs 0.070), does better on test: AP 0.090, difference with the rule +0.049 (+0.033 to +0.065), and 10.4% precision at 10%. It is not substituted afterwards: choosing it now would turn the test block into a second selection set. The likely cause is the noisy selection described above (274 late examples, drawn from a disrupted period). This is a property of the evaluation design to keep, not an anomaly to correct.

Performance moves with the late rate. Ranking quality is far from uniform over the test weeks (Figure 11). In May 2018, with a 6.5% late rate, the selected model's AP is 0.138; in June, when only 1.2% of orders were late, it falls to 0.034 (July: 0.048 at 3.2%). Rare-event months leave little signal to find, and 14 weeks give wide intervals. The results should be read as an order of magnitude, not as a stable operating characteristic.

Calibration does not transfer across seasons. Class weighting makes the raw logistic scores far too high (Brier 0.2520). The sigmoid calibrator [3, 4] learned on the calibration block (11,178 orders, 11.1% late) brings the Brier score to 0.0455, which is still worse than the constant prevalence (0.0364). Because the calibration months were about 3.2 times as risky as the test months, calibrated probabilities over-estimate risk (Figure 12). Rankings are unaffected, so the score can be used to *order* orders. To read it as a probability, it must first be recalibrated on recent deliveries.

Sensitivity and interpretation. The catalog attributes (category, weight, volume) come from a final product snapshot and are only *assumed* to be known at approval. Retraining without them leaves AP unchanged (0.075 for logistic regression, 0.092 for LightGBM), so the results do not rest on that assumption. Permutation importance on the calibration block (Figure 13) ranks seller-history support, customer state, promised lead time, same-state and distance features first. Consistent with Section 4, these are associations within this dataset and do not identify causes.

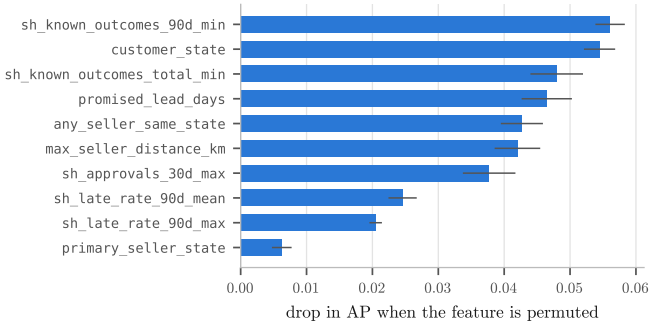


Figure 13: Permutation importance of the selected model (AP drop, 5 repeats) on the calibration block, never on test.

Table 5: Daily review policy. "Known" is the share of selected orders with an observable outcome; "Late" is computed among those; "Captured" is the share of all known late orders reviewed.

Ranking	q	Reviewed	Known	Late	Lift	Captured
Constant	5%	5.2%	98.0%	2.3%	0.62	3.2%
Constant	10%	10.2%	98.3%	3.1%	0.83	8.5%
Constant	20%	20.2%	98.3%	3.4%	0.90	18.2%
Seller rule	5%	5.2%	97.0%	4.6%	1.22	6.3%
Seller rule	10%	10.2%	97.6%	5.4%	1.43	14.5%
Seller rule	20%	20.2%	98.2%	4.8%	1.28	25.8%
Logistic regression	5%	5.2%	97.8%	8.1%	2.14	11.1%
Logistic regression	10%	10.2%	97.8%	8.2%	2.17	22.0%
Logistic regression	20%	20.2%	98.0%	7.2%	1.90	38.2%

8 From Scores to a Review Policy

The global top-q above ranks all labelled test orders at once, which is right for comparing models but not how a team works. The operational simulation replays each historical day: all orders approved that day are ranked by the score frozen at approval, and the top $\lceil qN_{\text{day}} \rceil$ are *selected before any outcome exists*. Only afterwards are outcomes looked up; some selected orders never get one (canceled or undelivered), and this share is reported rather than dropped. The rounding makes the reviewed share slightly larger than q (10.2% for $q = 10\%$, with a median of 197 approvals per day).

With a 10% daily capacity, the selected model puts 8.2% late orders in the review queue, against 5.4% for the seller-history rule and 3.8% on average (a lift of 2.17), and the queue contains 22.0% of all known late orders (14.5% for the rule). Doubling the capacity to 20% raises the share captured to 38.2% (Table 5, Figure 10). Only 2.2% of the selected orders have no observable outcome, so the selection bias from undelivered orders is small on this window.

These figures describe *targeting*. They do not show that reviewing an order changes its outcome, and no saving or delay reduction is claimed.

9 SQL Performance

Three representative queries were measured on the real volume in an isolated schema, with one warm-up and five timed runs on PostgreSQL 17.11 (Docker, 8 vCPUs, 4 GB). Each query was measured before and after an index or a rewrite motivated by its `EXPLAIN (ANALYZE, BUFFERS)` plan, and the results were checked to be identical (Table 6).

Table 6: Server execution time (median of 5, ms) and index build time (ms). Q1: as-of history for 99,991 targets against 97,733 events; Q2: monthly overview aggregation; Q3: one day of the risk review among 133,636 stored scores.

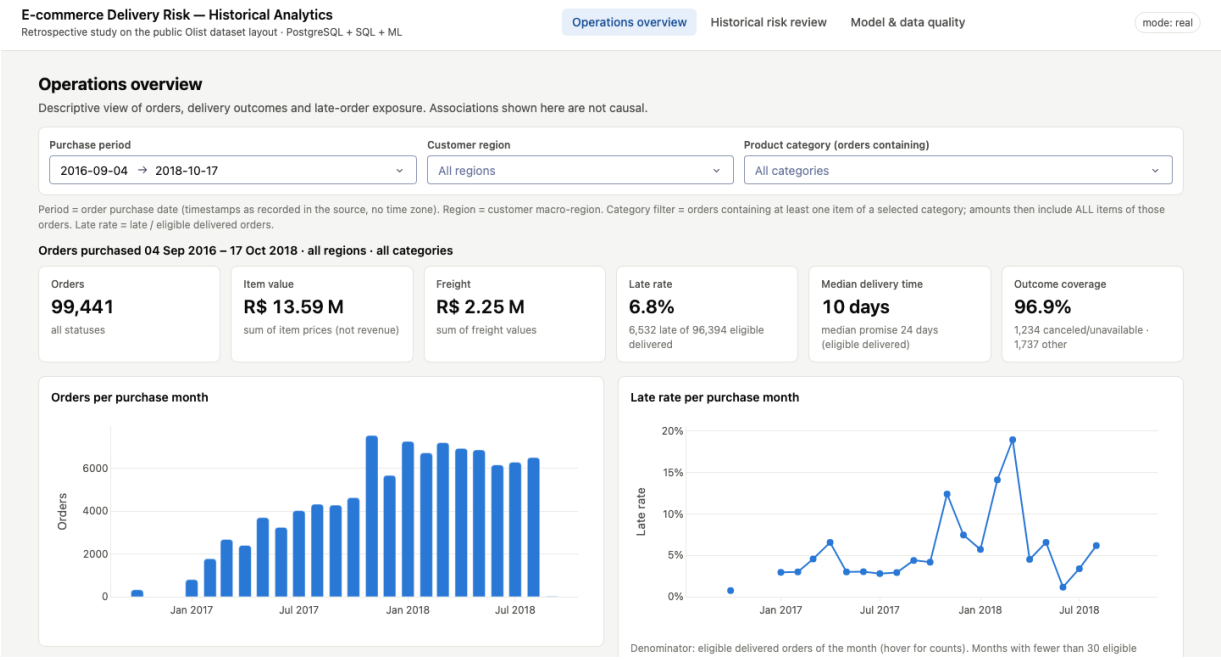
	Variant	Change	Time	Build
Q1	before	LATERAL join, no index	>300 000 (timeout)	–
Q1	rewrite	window functions over event streams, no index	11,321.1	–
Q1	after	LATERAL join + composite indexes	4,191.3	384
Q2	before	no index	18.0	–
Q2	after	index on order_categories (category, order_id)	13.3	153
Q3	before	no index	10.5	–
Q3	after	composite index matching filter + order	0.2	336

Reading the plans. For Q1, the `LATERAL` subquery runs once per distinct (seller, t). Without an index each execution scans the whole event table, roughly 10^5 targets times 10^5 events, and the query was canceled after 300 s. The composite index `(seller_id, outcome_known_at) INCLUDE (is_late, ...)` turns that scan into an index-only range scan of one seller's past (4.2 s). A rewrite as a single event stream with `RANGE BETWEEN INTERVAL '90 days' PRECEDING AND CURRENT ROW EXCLUDE GROUP` [9] needs no index and returns the same rows, but takes 11.3 s here. With frame exclusion, PostgreSQL can no longer maintain the window aggregate incrementally and re-aggregates every frame, which is costly for sellers with long histories. Q3 changes from a sequential scan plus sort (10.5 ms) to an index-only scan already in rank order (0.22 ms), so its index was kept. For Q2, a sequential scan with a hash join already answers in 18.0 ms; an index saved 5 ms and was not added. Query time is measured separately from rendering: the overview page takes 251 ms to render with a cold cache and 32 ms with a warm one.

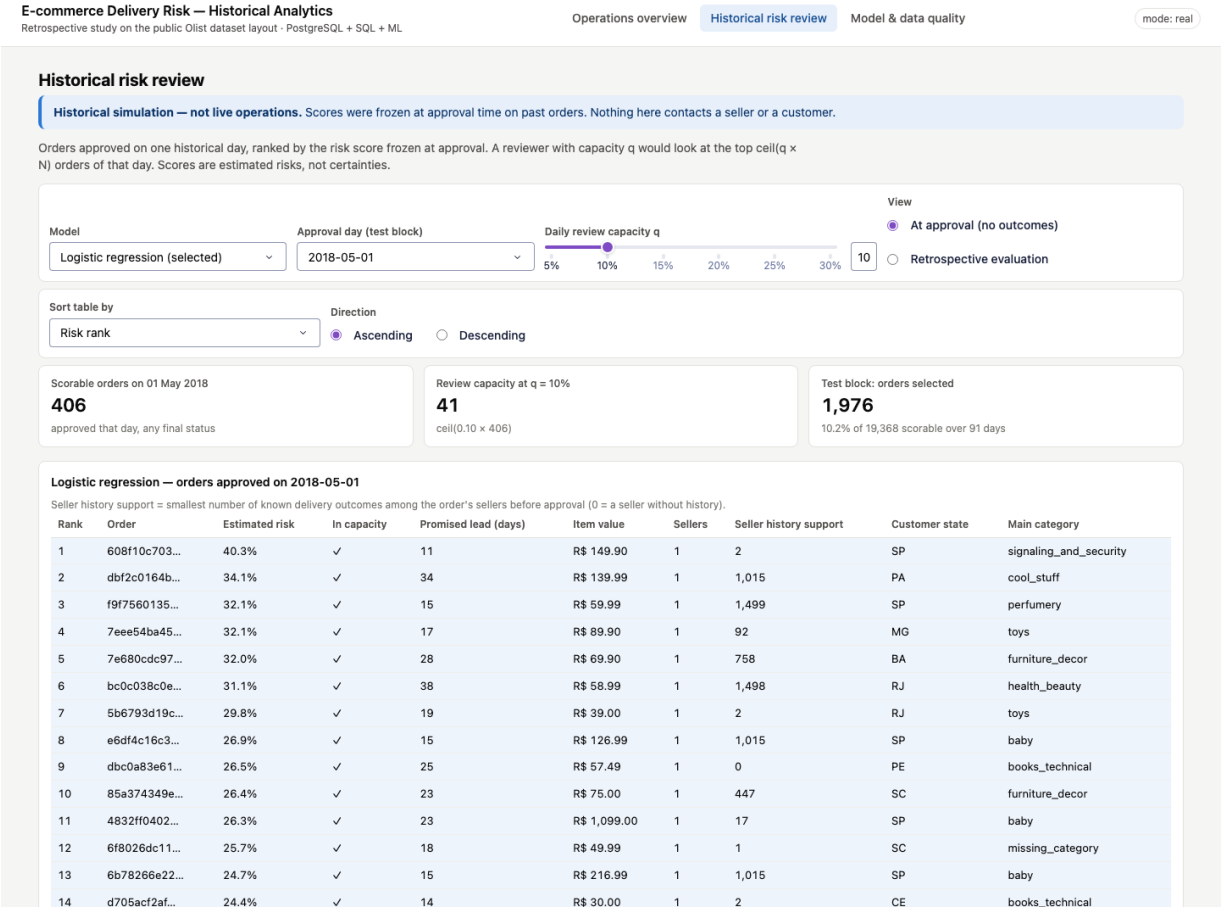
10 The Application

The Dash application reads the published tables through a read-only PostgreSQL role, in read-only transactions, with bound parameters only. Sort columns come from an allow-list, page sizes are capped, and results are cached under a key that includes the latest publication id, so a new pipeline run invalidates the cache. It has three pages (Figure 14):

- **Operations overview.** Period, region and category filters; the date convention is displayed. Each KPI shows its denominator. Category filtering selects orders *containing* a category, counted once. Segments and sellers below 30 eligible orders are hidden.



(a) Operations overview, all filters off: KPIs with their denominators and monthly trends.



(b) Historical risk review in *At approval* mode: one approval day ranked by frozen score; the capacity slider sets how many orders are flagged; no outcome is queried in this mode.

Figure 14: The application on the real data (screenshots at 1440 px; the pages were also checked at 1024 px).

- **Historical risk review.** One approval day of the test block ranked by frozen score, with a capacity slider. An *At approval* mode never queries outcomes; a separate *Retrospective evaluation* mode adds them with an explicit warning. A permanent banner states that this is a historical simulation, not live operations.
- **Model & data quality.** Model comparison with intervals, calibration, results by month and group (hidden below 30 orders or 10 late orders), split dates, data-quality checks and exclusions.

The application runs locally or in Docker (gunicorn). It is not deployed publicly. Doing so would require a hosting choice, a reduced database dump containing only the published schemas, TLS to the database, and a visible CC BY-NC-SA attribution.

11 Limitations and Next Experiment

- **Snapshot data.** Promise, prices and catalog attributes could have changed after approval without trace. The ablation shows that the catalog assumption does not drive the results; the promise assumption cannot be tested in this source.
- **Evaluated population.** Performance is measured on orders that were finally delivered. Scores of canceled or undelivered orders cannot be checked.
- **Few weeks, strong seasonality.** With 14 test weeks the intervals are wide, performance depends heavily on the month's late rate, and calibration does not carry over between periods.
- **Label cut-offs.** They are the right rule for an honest simulation, but here they left model selection with few late examples, and the choice it made was not the best on test.
- **Shared outcomes.** Multi-seller orders share one delivery timestamp, and whether it refers to the first or the last parcel is not documented.
- **No causal evidence.** A better-targeted queue is not proof that intervening helps.

Next experiment. The question that matters, whether contacting the seller of a risky order reduces lateness, requires an intervention. Each day, the top-q orders would be split at random into a *contacted* and a *not contacted* arm, so that both arms have the same score distribution. Comparing late rates, delivery delays and review scores between the arms over several weeks measures the effect of the action itself. The same pilot would collect the cost of each contact, which is needed before any economic claim. The model would be recalibrated on recent deliveries before it starts.

12 Conclusion

Most of the value of this project lies in what it refuses to count. Joins are checked against their totals, so a silent 4.0% inflation cannot reach a dashboard. Features only see deliveries that had happened, as an independent re-implementation confirms on every real row. The model is chosen before the test block is opened, and the result is kept even when a rejected model turns out better. On these terms, a simple logistic regression ranks risky orders measurably better than a seller-history rule: AP 0.075 vs 0.043, with a paired interval excluding zero. A 10% daily review would have worked on a queue 2.17 times richer in late orders. The gain is real but modest, and it depends on the season. Turning it into a decision requires recalibration and the randomised pilot described above.

References

[1] Olist, "Brazilian E-Commerce Public Dataset by Olist," Kaggle, version 2, 2021. CC BY-NC-SA 4.0.

[2] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," *PLoS ONE*, 10(3), 2015.

[3] J. Platt, "Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods," in *Advances in Large Margin Classifiers*, MIT Press, 1999.

[4] A. Niculescu-Mizil and R. Caruana, "Predicting Good Probabilities with Supervised Learning," *Proc. ICML*, 2005.

[5] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," *Advances in Neural Information Processing Systems*, 2017.

[6] H. R. Künsch, "The Jackknife and the Bootstrap for General Stationary Observations," *The Annals of Statistics*, 17(3), 1989.

[7] S. Kaufman, S. Rosset, C. Perlich and O. Stitelman, "Leakage in Data Mining: Formulation, Detection, and Avoidance," *ACM Transactions on Knowledge Discovery from Data*, 6(4), 2012.

[8] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, 12, 2011.

[9] PostgreSQL Global Development Group, *PostgreSQL 17 Documentation*: "Window Functions" and "Using EXPLAIN," 2024.